DEPLOYMENT

Hosting the Agent SDK

Deploy the Agent SDK in production: subprocess architecture, session persistence, scaling, observability, and multi-tenant isolation for Docker, Kubernetes, and sandbox providers.

The Agent SDK spawns and supervises a `claude` CLI subprocess that owns a shell, a working directory, and session files on disk. Hosting it is not like hosting a stateless API wrapper. Every running agent is a long-lived process tied to local state, which shapes how you allocate resources, persist sessions, and scale across tenants.

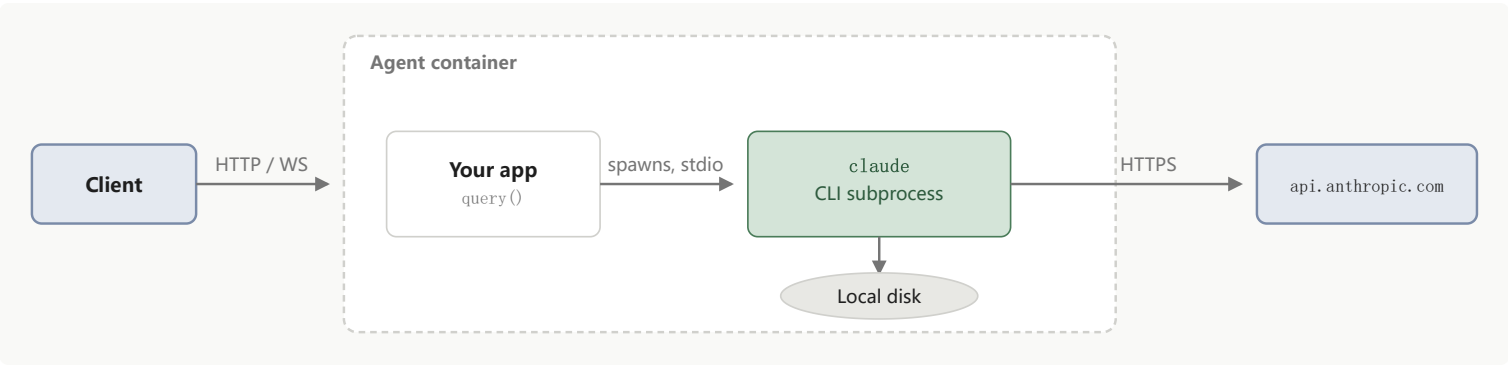
This page covers self-hosting on your own infrastructure: understand **the subprocess model**, **choose a session pattern**, **provision the container**, and **handle production concerns** like persistence, observability, auth, and multi-tenant isolation. For deployable Dockerfiles and Kubernetes manifests, see the **hosting cookbook**.

If you do not need infrastructure control, custom isolation, or your own data plane, consider **Managed Agents** instead: a hosted REST API where Anthropic runs the agent and the sandbox, so your application sends events and streams back results with no hosting infrastructure to operate.

> For security hardening beyond basic sandboxing, including network controls, credential management, and isolation options, see **Secure Deployment**.

The subprocess model

Every hosting decision on this page follows from how the SDK runs the agent. When your code calls `query()`, the SDK spawns a separate `claude` CLI process and talks to it over stdio. That subprocess owns the shell, the working directory, and the JSONL session transcripts on local disk.



One agent session maps to one subprocess. Running N concurrent sessions means N subprocesses, each with its own process tree and transcript file. By default they all inherit your application's working directory, so pass `cwd` on each `query()` call when sessions need separate filesystems:

```
query({ prompt, options: { cwd: "/work/session-a" } })
```

State that lives on local disk

Three kinds of agent state live on the container's filesystem by default. None of them survive a container restart, a scale-down, or a move to a different node.

State	Default location
Session transcripts	`~/.claude/projects/`, or the `projects/` directory under `CLAUDE_CONFIG_DIR` if set
`CLAUDE.md` memory files	`~/.claude/CLAUDE.md` for the user tier and the session's working directory for the project tier
Working-directory artifacts	The session's working directory

To persist transcripts across hosts, configure a `SessionStore` **adapter**. Memory files and other working-directory artifacts need their own storage strategy, such as a mounted volume or an object-store sync.

For how sessions, resumption, and forking work at the API level, see **Sessions**.

Choose a session pattern

These four patterns cover session lifecycle: how long a container lives relative to the sessions it serves. For where the container runs, the **hosting cookbook** has **deployable code** for local Docker,

Modal, and Kubernetes. Choose a session pattern here and a deployment target from the cookbook.

Ephemeral sessions

Create a container for each user task and destroy it when the task completes. Best for one-off tasks. The user may still interact with the AI while the task is completing, but once completed the container is destroyed.

Example workloads include bug investigation and fix, invoice and receipt extraction, document translation, and media transformation.

The container runs a one-shot entrypoint that calls the SDK and exits. The example below shows a minimal TypeScript version. Save it as `entrypoint.mts` or set `"type": "module"` in `package.json` so top-level `await` is available.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

const prompt = process.env.TASK_PROMPT!;
for await (const message of query({ prompt, options: { maxTurns:
20 } })) {
  console.log(message);
}
```

Long-running sessions

Run persistent container instances, often hosting multiple SDK processes per container, to serve ongoing work. Best for agents that take autonomous action, serve content, or handle high-volume message streams.

Example workloads include an email agent that triages and responds to incoming mail, a site builder that hosts a per-user editable site through container ports, and a chat bot that handles continuous traffic from a platform like Slack.

The container exposes an HTTP or WebSocket endpoint and maps each active session to a long-lived query and the subprocess behind it. In TypeScript, use `streamInput()` to add turns to an active session and `startup()` to pre-warm subprocesses ahead of incoming traffic. In Python, use `ClaudeSDKClient` to hold a session open across turns. Size the container so it can hold the maximum number of concurrent sessions in memory.

Hybrid sessions

Ephemeral containers that hydrate from a `SessionStore` on startup and persist updates back. Best for sessions that span many interactions but sit idle between them. The container spins down during idle periods and spins back up when the user returns.

Example workloads include a personal project manager with intermittent check-ins, deep research that pauses and resumes over hours, and a customer support agent that loads ticket history across interactions.

Tune your provider's idle timeout to how frequently you expect users to return. Shutting a container down without a `SessionStore` configured loses the transcript with it, so the store is required for this pattern, not optional.

The pattern hinges on resuming a session by ID with a shared store attached:

```
import { query, type SessionStore } from "@anthropic-ai/claude-agent-sdk";

declare const userInput: string;
declare const sessionId: string;          // looked up from your database by
user
declare const sessionStore: SessionStore; // S3, Redis, Postgres, or your own
adapter

for await (const message of query({
  prompt: userInput,
  options: { resume: sessionId, sessionStore },
})) {
  // ...
}
```

See **Session storage** for the full `SessionStore` interface and reference adapters.

Multi-agent container

Run multiple SDK subprocesses inside one container. Best for agents that must collaborate closely, for example multi-agent simulations where the agents interact with each other in a shared environment.

Give each agent its own working directory so they do not overwrite each other's files, and isolate settings loading so per-agent `CLAUDE.md` files do not leak across agents. See **Multi-tenant isolation**

for the specific options.

Provision the container

Container-based sandboxing

Run the SDK inside a sandboxed container for process isolation, resource limits, network control, and an ephemeral filesystem. Several providers specialize in sandboxed container environments that fit the Agent SDK's model.

Questions to answer when choosing a provider:

- **Who runs the sandbox**: a sandbox-as-a-service provider operates the infrastructure for you, while self-hosted options give you software to run on your own.
- **Cold-start latency**: how long from "create a sandbox" to "ready to accept the first request." Ephemeral patterns need sub-second starts. Long-running patterns tolerate more.
- **Persistent storage**: whether the provider offers durable volumes or only ephemeral disk. The hybrid pattern needs durable storage somewhere, whether in the sandbox or alongside it.
- **Pricing model**: per-second, per-request, or flat hourly billing. Per-second pricing suits bursty ephemeral workloads. Hourly suits long-running sessions.
- **Networking**: support for custom egress rules, outbound proxies, and private VPC peering for regulated environments.

Providers to evaluate:

- **Modal Sandbox**, with a **demo implementation**
- **Cloudflare Sandboxes**
- **Daytona**
- **E2B**
- **Fly Machines**
- **Vercel Sandbox**

For self-hosted options such as Docker, gVisor, and Firecracker, and detailed isolation configuration, see **Isolation Technologies**.

Runtime dependencies

The container needs only your SDK's language runtime:

- Python 3.10+ for the Python SDK, or Node.js 18+ for the TypeScript SDK
- Both SDK packages bundle a native Claude Code binary for the host platform, so no separate Claude Code or Node.js install is needed for the spawned CLI

The bundled binary is pinned to the SDK package version, so updating the SDK is how you update the CLI. The SDK follows semver: take patch releases continuously and review the **TypeScript** or **Python** changelog before taking a minor.

Resources

1 GiB RAM, 5 GiB disk, and 1 CPU per agent is a reasonable starting point for a freshly started instance. Memory usage grows with session length and tool activity, so size for the session lengths and concurrency you actually need rather than the idle baseline. See **Scaling and concurrency** for how to work out agents per host.

Network

The SDK needs outbound HTTPS to `api.anthropic.com`, or to your provider's regional endpoint when running on Bedrock or Vertex. If your agents use **MCP servers** or external tools, they need outbound access to those endpoints as well. For production, route outbound traffic through an egress proxy that enforces domain allowlists, injects credentials, and logs requests. See **Secure Deployment** for the full pattern.

For inbound traffic, expose an HTTP or WebSocket port on the container. Your application handles client requests on that port and calls the SDK internally; the subprocess itself does not listen on the network.

Handle production concerns

Work through these decisions before shipping a self-hosted agent.

Session and state persistence

Default local disk is lost on restart, scale-down, or a move to a different node. For any session a user expects to resume, mirror the transcript to durable storage with a `SessionStore` **adapter**. See

Reference implementations for S3, Redis, and Postgres adapters and a conformance suite for your own.

Three things to know about how `SessionStore` behaves:

- **Transcripts only**: `SessionStore` mirrors transcripts, not `CLAUDE.md` memory files or other working-directory artifacts. Mount a shared volume or sync those separately.
- **Mirror, not replacement**: the subprocess writes to local disk first, and the store receives a copy of each batch. Local writes remain authoritative.
- `mirror_error` **messages**: if the store rejects or times out, the SDK emits a `{ type: "system", subtype: "mirror_error" }` message and continues the query without retry. Alert on these if store durability matters.

Observability

Agent SDK agents are long-lived processes that spawn tool calls across many API round-trips. Without telemetry you cannot see which tools ran, how long they took, or where a session stalled.

The SDK inherits OpenTelemetry configuration from the environment. Set the OTEL environment variables at the container or orchestrator level so every `query()` call exports spans, metrics, and log events to your collector. The example below enables OTLP export for all three signals. `CLAUDE_CODE_ENHANCED_TELEMETRY_BETA` is required only for traces; omit it if you export metrics and logs alone.

.env

```
CLAUDE_CODE_ENABLE_TELEMETRY=1
CLAUDE_CODE_ENHANCED_TELEMETRY_BETA=1
OTEL_TRACES_EXPORTER=otlp
OTEL_METRICS_EXPORTER=otlp
OTEL_LOGS_EXPORTER=otlp
OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
OTEL_EXPORTER_OTLP_ENDPOINT=http://collector.example.com:4318
```

Prompt text and tool inputs are not included in exports by default. See **Control sensitive data in exports** for the opt-in flags, and **Observability** for the full signal catalog.

Auth and secrets

Three auth concerns matter at hosting time:

- **Anthropic API**: the subprocess reads `ANTHROPIC_API_KEY` from its environment. Supply it from your secret manager, or set `ANTHROPIC_BASE_URL` to route model calls through a proxy that injects the key outside the container. See **Credential management** for the proxy pattern and the **SDK overview** for supported authentication methods.
- **Inbound**: put authentication at a gateway in front of the agent container. The agent should receive pre-authenticated requests and should not be the component that validates user tokens.
- **Outbound tools**: keep tool credentials out of the agent environment. Route outbound calls through a proxy that injects API keys after the request leaves the container. The agent makes the call; the proxy adds the credential.

Scaling and concurrency

Each session runs in its own subprocess, so concurrency on a host is bounded by how many subprocesses its RAM can hold.

Size each host with this formula:

```
agents per host = (host RAM - overhead) / (per-session RAM
ceiling)
```

Measure the per-session ceiling by running a representative session to your target length under your expected tool load and recording peak RSS. The 1 GiB starting point in **Resources** is a floor, not the ceiling.

Horizontal-scale routing depends on your pattern. For long-running sessions, where containers hold many sessions, run a pool of containers behind a load balancer and pin each session to one container using consistent hashing on `sessionId`. A pinned session keeps hitting the same container, and therefore the same running subprocess, until it is evicted or the container restarts.

Large fanouts of concurrent **subagents** from a single session can hit API rate limits. Break the work into smaller batches rather than issuing one wide dispatch.

Cost

Anthropic token cost typically dominates container infrastructure cost by an order of magnitude or more. A minimally provisioned container runs roughly $0.05 per hour, while a single long agent session can spend dollars in tokens. See **Cost tracking** for per-session token accounting.

Multi-tenant isolation

Default SDK behavior reads settings and `CLAUDE.md` memory files from the filesystem. In a shared container that serves multiple tenants, those files can leak one tenant's context into another tenant's session.

To isolate tenants inside a shared container:

- Pass `settingSources: []` in TypeScript or `setting_sources=[]` in Python so no filesystem settings load.
- Set `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1` in `env`. **Auto memory** at `~/.claude/projects/<project>/memory/` loads into the system prompt regardless of `settingSources`. See **What settingSources does not control** for the other inputs that load unconditionally.
- Point `CLAUDE_CONFIG_DIR` at a per-tenant directory so tenants do not share the `~/.claude.json` global config.
- Use a per-tenant working directory. Pass `cwd` explicitly on every `query()` call.
- Apply per-tenant egress rules at your proxy, such as distinct outbound IPs, credentials, or domain allowlists, so a compromised tenant cannot exfiltrate data via another tenant's outbound policy.

The example below applies the four SDK-level options together. Construct `tenantDir` and `configDir` so each tenant gets a path no other tenant can read. In TypeScript, `env` replaces the subprocess environment, so spread `...process.env` to keep inherited variables like `PATH` and `ANTHROPIC_API_KEY`. In Python, `env` is merged on top of the inherited environment.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

declare const prompt: string;
declare const tenantDir: string;
declare const configDir: string;

for await (const message of query({
  prompt,
  options: {
    cwd: tenantDir,
    settingSources: [],
    env: {
      ...process.env,
      CLAUDE_CONFIG_DIR: configDir,
      CLAUDE_CODE_DISABLE_AUTO_MEMORY: "1",
    },
  },
})) {
  // ...
}
```

For per-tenant network controls, see **Secure Deployment**.

Known limitations

Plan around these in your deployment design.

Limitation	What to do
No top-level session timeout	A session does not time out on its own. Set `maxTurns` in `Options` to bound how many tool-use round trips the agent takes before stopping.
Memory growth over long sessions	Cap session length or recycle subprocesses periodically. See **Scaling and concurrency**.
Large parallel-subagent fanouts can hit rate limits	Break work into smaller batches rather than issuing one wide dispatch.
No per-subagent wall-clock deadline	Cap each **subagent** with `maxTurns` in its `AgentDefinition`. For background subagents only, `CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS` sets a stall watchdog that fires when a `run_in_background` subagent stops producing output; it is not a total-runtime deadline.

Next steps

- **Hosting cookbook**: notebook walkthrough with **deployable code** for Docker, Modal, and Kubernetes.
- **Session storage**: persist transcripts across hosts with a `SessionStore` adapter.
- **Observability**: export OTEL traces, metrics, and logs to your collector.
- **Secure deployment**: network controls, credential management, and isolation hardening.
- **Cost tracking**: per-session token and cost accounting.